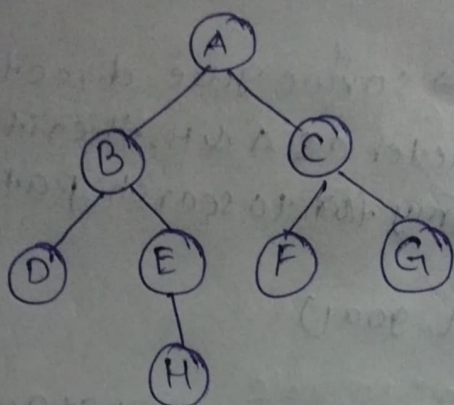


lab-2a: Breadth-First Search

Given tree.



Thing to find:

1. Find a path from A to H using BFS
2. Return the path as list
3. Return the order in which nodes are visited
4. Diff b/w BFS & DFS.

BFS: Level-by-level

Level-1 $\rightarrow$ A

Level-2 $\rightarrow$ B, C

Level-3 $\rightarrow$ D, E, F, G

Level-4 $\rightarrow$ H

BFS: explores nodes level-by-level

EX:

A $\rightarrow$ B, C $\rightarrow$ D, E, F, G $\rightarrow$ H.

↓
level-by-level

graph = { 'A': ['B', 'C'],

'B': ['D', 'E'],

'C': ['F', 'G'],

'D': [],

'E': ['H'],

'F': [], 'G': [], 'H': [] }

- declaring the graph as a dictionary.

if there is no children for the node then we declare that as empty.

def bfs(start, goal)

↓
this function is

bfs('A', 'H') → so we give directly the parameter as A & H, then it would work only for to search path from A to H.

so, bfs(start, goal)

↓
This is useful to give any start node and goal node and then can find the path b/w those 2 nodes.

- We use Queue data structure here, so that the nodes can be traversed level wise.

Queue = [start]

↓
initially putting the start node and then starting traversing from there level-by-level.

- we use, visited = []

↓
to maintain all the visited nodes, so that there is no chance traversing the same node again.

- We ~~find~~ traverse the nodes, until the element is found or else until queue not empty.

queue.pop(0) ↓

This is the line that makes it different from DFS which means, removes the element from first

Queue - First in First out.

- If the goal is found, ↓

return path & visited nodes list.

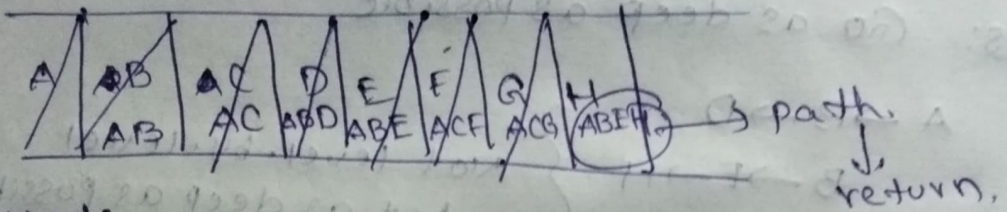
- if the current node is already not visited, then we add that node to the visited list and then that node's corresponding children nodes will

be added to the queue.
 - if we came out of the loop without finding our goal element node, then the node is not found in graph.

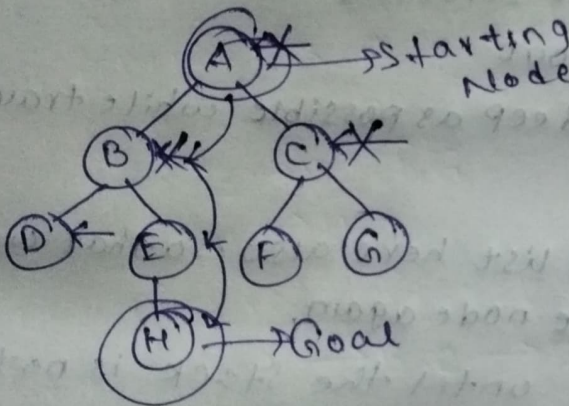
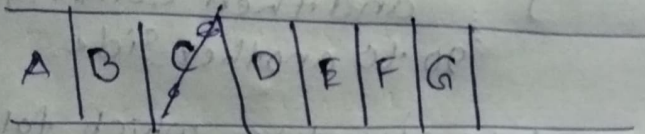
↓
 So, we return None, and only return the paths we visited or nodes we visited.

Dry Run:

Queue:



visited:



Diff b/w BFS & DFS:

BFS → queue.pop(0) → removes first element

DFS → queue.pop() → removes last element

